

The Core Principle

Build the demo path, not the full system. Judges see the screen, not the code.

You need exactly 3 things working on Day 3:

- 1. A borrower's data goes in
- 2. A GCI score + temporal report comes out
- 3. It looks professional on a dashboard

Everything else is secondary.

AI Agent Strategy — Maximizing Free Limits

You have access to multiple free AI tools. Split the work across them so no single one hits its limit:

Agent	Best Used For	Free Limit Strategy
Claude (this)	Architecture decisions, complex logic, debugging	Use for hard problems only
ChatGPT free	Boilerplate code, repetitive functions	Use for scaffolding
GitHub Copilot	In-editor autocomplete while coding	Free for students
Cursor AI	Whole-file generation	Free tier 50 uses/day
Google Gemini	Documentation, data generation scripts	Generous free tier
v0.dev (Vercel)	React dashboard UI generation	Free tier available

Rule: Never write boilerplate manually. Always prompt an AI first.

Day 1 — Data + ML Core (The Brain)

Morning (4 hours) — Synthetic Data

This is your foundation. Everything depends on it.

Prompt Gemini or ChatGPT with this exact request:

Generate a Python script using Faker, NumPy, and Pandas that creates 500 synthetic gig worker borrower profiles with 300-600 transactions each. Include festival seasonality (Diwali October spike +40%, IPL April spike +25%, Monsoon July dip -20%), 5 manipulation patterns (income spiking, round-trip transactions, expense suppression, synthetic regularity, dormant activation) injected in 20% of profiles. Output: gig_transactions.csv with fields: transaction_id, borrower_id, timestamp, amount, direction, counterparty_id, counterparty_type, payment_mode, category, balance_after_txn, festival_window

Run it. Verify the CSV looks right. This one script saves you 2 days.

Afternoon (4 hours) — Feature Engineering + Models

Use Claude for this — it's the complex logic part.

build ml/features/feature_engineering.py —
takes cleaned CSV, outputs feature matrix with:
avg_monthly_income, income_volatility, cvd_residual_variance,
liquidity_buffer_weeks, expense_income_ratio, round_trip_count,
dormancy_score, weekly_std_ratio, festival_uplift_ratio

Then train in sequence — use ChatGPT for boilerplate:

```
python  
# M1 — STL Decomposition (statsmodels)  
# M2 — Isolation Forest (sklearn)  
# M3 — Hard rules (pure Python if/else)  
# M4 — XGBoost classifier
```

Day 1 End Goal: `python train_all.py` runs without errors. Four model files saved in `/models/`.

Day 2 — Backend + Connect Everything (The Spine)

Morning (3 hours) — FastAPI Backend

Prompt Cursor or ChatGPT:

Create a FastAPI app with these 5 endpoints:
POST /upload-transactions (accepts CSV, runs full ML pipeline)
GET /gci/{borrower_id} (returns GCI score + all sub-scores)
GET /temporal/{borrower_id} (returns decomposed income arrays)
GET /anomaly/{borrower_id} (returns flags + anomaly score)
GET /borrowers (returns list of all borrower IDs)
Use SQLite for MVP (not PostgreSQL — simpler setup).
Load ML models from /models/ at startup.
Return clean JSON.

SQLite instead of PostgreSQL saves you 2 hours of setup. Swap it later.

Afternoon (2 hours) — Pipeline Connection

This is the critical wiring step. When a file uploads:

CSV upload → clean_data() → engineer_features() →
run_stl() → run_isolation_forest() → run_rules() →
run_xgboost() → calculate_gci_score() → save_to_db() → return JSON

Test with 3 borrowers manually via Postman or curl. Fix until it works.

Late Afternoon (2 hours) — Docker

dockerfile
One docker-compose.yml that starts:
- FastAPI on port 8000
- React on port 3000
Run: docker compose up

Use ChatGPT to generate this. Takes 20 minutes, saves demo day disasters.

Day 2 End Goal: `curl localhost:8000/gci/borrower_001` returns a real JSON with a GCI score.

Day 3 — Frontend + Polish (The Face)

Morning (3 hours) — React Dashboard with v0.dev

This is your secret weapon. Go to v0.dev and prompt:

Create a React financial dashboard with two tabs:
Tab 1 "Risk Analysis": anomaly score gauge (0-1),
5 fraud flag checklist (green/red),

bar chart of monthly income with red highlighted suspicious months,
manipulation risk label (Clean/Suspicious/High Risk)

Tab 2 "GCI Report": large GCI score display (300-900),
line chart showing income decomposition (raw vs seasonal vs residual),
donut chart showing variance breakdown (festival% / weekly% / residual%),
3-bar stress test chart (Normal/25%drop/40%drop repayment probability),
horizontal bar chart for feature importance (SHAP values),
lender recommendation card

Use Tailwind CSS. Professional fintech dark theme.
Fetch data from localhost:8000

v0.dev generates the entire React component. You copy-paste, connect to your API, done.

Afternoon (2 hours) — Wire Frontend to Backend

Replace mock data in v0.dev output with real API calls:

```
javascript
// TanStack Query or simple fetch
const { data } = useQuery(['gci', borrowerId],
  () => fetch(`http://localhost:8000/gci/${borrowerId}`).then(r => r.json())
)
```

Add a borrower selector dropdown populated from `/borrowers` endpoint. Make sure Raju's profile (with a good score ~760) and a fraud profile (High Risk) are both in your demo dataset.

Late Afternoon (1 hour) — The Demo Script

This is what wins. Prepare exactly this flow:

1. Open dashboard → select "Raju Patil"
2. Show Tab 2 → GCI Score 762, Diwali spike explained, Clean label
3. "Now watch what happens with a manipulated profile..."
4. Select "Vikram S." → Score 431, Income Spiking flag RED, High Risk
5. Show the festival camouflage detection → "our system caught that his October spike was 3x above the cohort ceiling"
6. Back to Tab 2 → Stress test → "even under 25% income drop, Raju still repays with 78% probability"

Practice this 3 times. Know it cold.

The Non-Negotiable Shortcuts

These save you from rabbit holes that kill 3-day builds:

- Use **SQLite not PostgreSQL** — same Python API, zero setup
 - Use **synchronous FastAPI** — no Celery, no Redis, no async complexity
 - Use **pre-computed features** — run feature engineering once, save to CSV, load at startup
 - Use **3 hardcoded demo borrowers** — Raju (clean), Vikram (fraud), one borderline — makes demo reliable
 - Skip **JWT auth entirely** — open API for demo, add later
 - Skip **real AA integration** — CSV upload is enough for demo
 - Use **joblib.load()** not MLflow — simpler model loading
-

Hour-by-Hour Fallback Priority

If you're running behind, cut in this order:

Cut first	Keep always
Docker setup	GCI score calculation
Redis caching	Temporal decomposition chart
Batch scoring endpoint	Anomaly flag panel
Cohort percentile system	Raju vs Vikram contrast demo
SHAP explainability	Fraud detection story

The minimum viable demo is: one borrower profile, one GCI score, one chart showing the seasonal decomposition, and one fraud flag firing red. That alone tells the story.

Your Day 3 End State

localhost:3000 — React dashboard, two tabs, live data
localhost:8000/docs — FastAPI auto-docs showing all endpoints
3 demo borrowers loaded — clean, fraud, borderline
The Raju story told in under 2 minutes on screen

When showcasing your prototype, you are effectively demonstrating the **proof of concept** for the larger vision. Based on your prompts and the **GCI Architectural Blueprint**, you can confidently guarantee that the MVP implements the **functional logic** and **mathematical core** of the full version, even if the underlying infrastructure is simplified.

Here is what you can explicitly guarantee during your presentation:

1. Implementation of the "Four-Model Pipeline"

You can claim that the MVP is not a simple heuristic but a sophisticated pipeline where every request passes through all four specialized stages:

- **M1 (Temporal Decomposer):** Active use of **STL Decomposition** to separate trend, seasonal, and residual components.
- **M2 (Anomaly Detector):** An **unsupervised Isolation Forest** model scoring borrower feature vectors.
- **M3 (Fraud Rule Engine):** Hard statistical checks for the five core flags (F1–F5).
- **M4 (PD Scorer):** A supervised **XGBoost** model predicting the final Probability of Default.

2. The "CVD" Framework (Your Core Innovation)

You can guarantee that the MVP successfully performs **Causal Volatility Decomposition**. This means the system is mathematically capable of:

- Identifying and "forgiving" volatility caused by the **India Festival Calendar** (Diwali, IPL, etc.).
- Calculating the **explained variance ratio** to determine how much of a borrower's income swing is "natural" versus "stochastic risk".

3. Comprehensive Feature Engineering (RBI-Aligned)

You can state that the features being calculated are not arbitrary but are grounded in the **RBI Digital Lending Guidelines (2022)** and the **Account Aggregator framework**. You are using:

- **Liquidity & Repayment Features:** Such as the Liquidity Cover Ratio (LCR) and EMI Coverage Ratio.
- **Temporal Features:** Such as income trend slope and seasonal amplitude.
- **Anomaly Features:** Including round-trip pair counts and dormancy scores.

4. Regulatory-Grade Explainability (SHAP)

You can guarantee that the GCI score is **interpretable**, not a "black box." The MVP uses **SHAP (SHapley Additive Explanations)** to extract the top 5 drivers for every score. This ensures the prototype is already compliant with the transparency requirements of Section 7.3 of the RBI guidelines.

5. Automated Fraud Detection (The F1–F5 Flags)

You can demonstrate that the system is pre-configured to catch the **5 specific manipulation patterns** identified in the blueprint:

- **Income Spiking (F1)** and **Expense Suppression (F2)**.
- **Round-trip transactions (F3)**, **Synthetic regularity (F4)**, and **Dormant activation (F5)**.

6. Cohort-Relative Scoring Logic

You can claim the system uses **micro-cohorts** (based on occupation, city tier, and income band). The prototype does not judge a delivery rider in Pune by the same standard as a freelancer in Mumbai; every anomaly threshold is cohort-relative.

Comparison Summary for your Slide

Feature	Status in MVP (Prototype)	Guarantee for Full Version
Data Source	Synthetic Gig Worker Profiles	Live Bank/UPI via Account Aggregator
Model Pipeline	Full 4-Model logic implemented	Continuous retraining & GNN/LSTM upgrades
Explainability	Individual SHAP values per report	Full audit trails & automated compliance docs
Database	SQLite (for rapid validation)	PostgreSQL 16+ for massive scale
Inference	Local Celery/FastAPI execution	High-availability AWS ECS Fargate cluster

Presentation Tip: When showing "Vikram S.," emphasize that the prototype has already caught a "seasonal camouflage" attempt—proving that your **M1 + M3** logic works exactly as specified in the research claim of the Architectural Blueprint